МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по учебной практике

ПМ11. Разработка, администрирование и защита баз данных

Студент

Шумайлов Кирилл Константиновчи

Группа 23П-2

Специальность 09.02.07 Информационные системы и программирование

Руководитель практики от колледжа:

Калинин Арсений Олегович

Руководитель практики от организации:

_____ *Калинин Арсений Олегович*

подпись

УТВЕРЖДАЮ:

Директор

_____

Наименование организации

Слободской колледж педагогики и социальных отношений

_____

_____/_____

подпись расшифровка

М. П.

2025-2026 уч. год

Содержание

Оглавление
- Выполнение анализа и предварительной обработки информации
- Выделение объектов и атрибутов в соответствии с заданием
- Проектирование и нормализация БД в полном соответствии с поставленной задачей и применением CASE-средств
- Выполнение построения БД в предложенной СУБД и заполнение всех таблиц с помощью соответствующих средств
- Реализация уровней доступа для различных категорий пользователей
- Создание запросов и отчетов в соответствии с заданием
- Создание и обоснование групп пользователей, принципов регистрации и системы паролей
- Выполнение резервного копирования БД и восстановление состояния БД на заданную дату
- Подготовка к защите отчета по практике.

Приложения

1. ВЫПОЛНЕНИЕ АНАЛИЗА И ПРЕДВАРИТЕЛЬНОЙ ОБРАБОТКИ ИНФОРМАЦИИ

На первом этапе практики был проведён анализ предметной области системы контроля пропускного режима. Выявлены основные бизнес-процессы, участники и информационные потоки.
Для формализации анализа использовались средства **UML 2.0** и **ER-моделирование**:

- Построена **диаграмма вариантов использования**, отражающая роли пользователей и основные функции системы (Рисунок 1).
- Разработана **диаграмма деятельности**, описывающая жизненный цикл заявки на посещение (Рисунок 2).
- Создана **концептуальная ER-диаграмма**, показывающая основные сущности и связи между ними на начальном этапе проектирования (Рисунок 3).

В процессе предварительной обработки информации были выявлены повторяющиеся группы данных, неатомарные атрибуты (ФИО, паспортные данные), а также определены правила валидации (возраст посетителя не младше 16 лет, ограничение срока заявки и др.).

1.2 UML Диаграмма вариантов использования

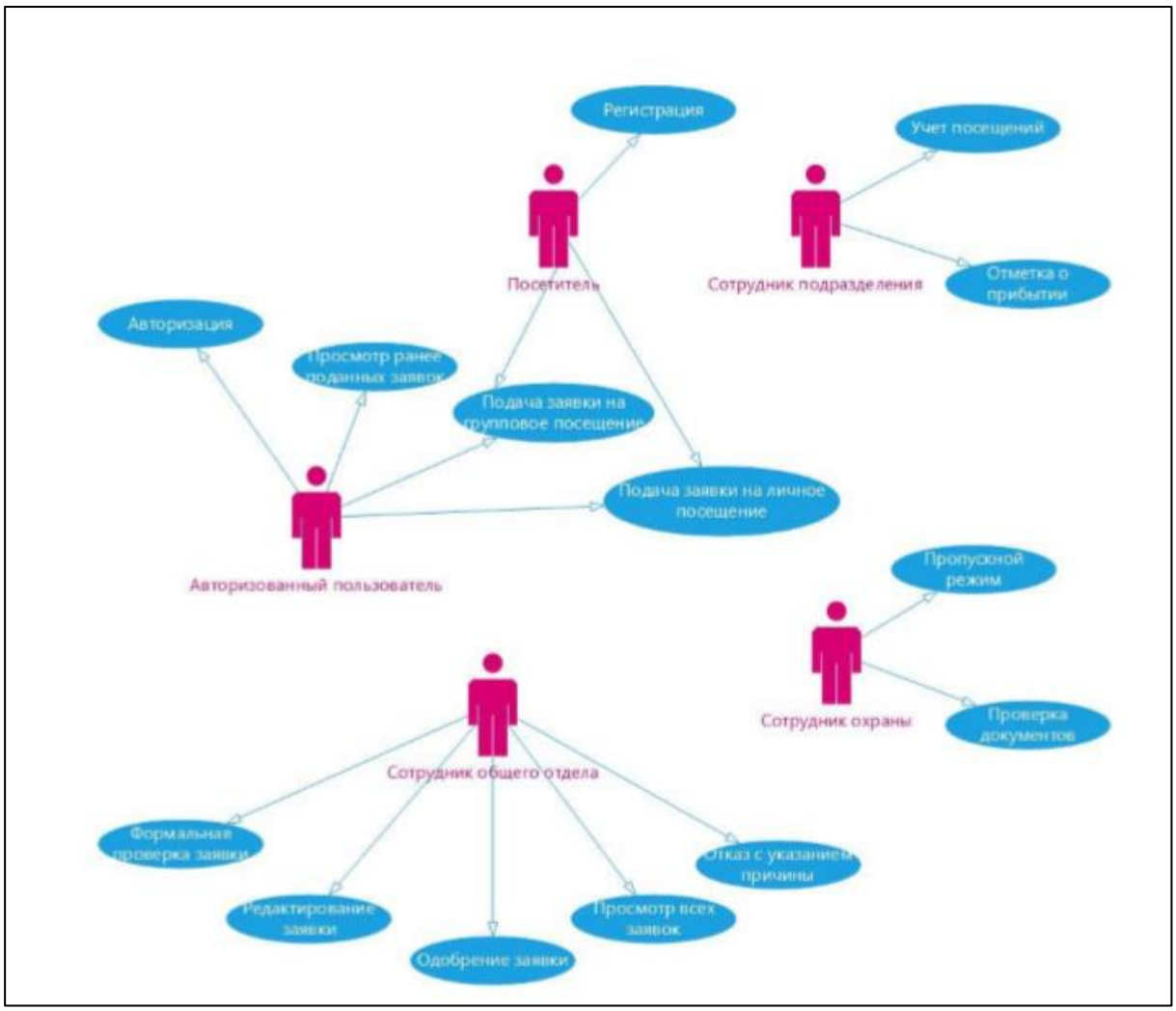

Рисунок 1 - UML Диаграмма вариантов использования

1.3 ERD диаграмма (Схема базы данных)

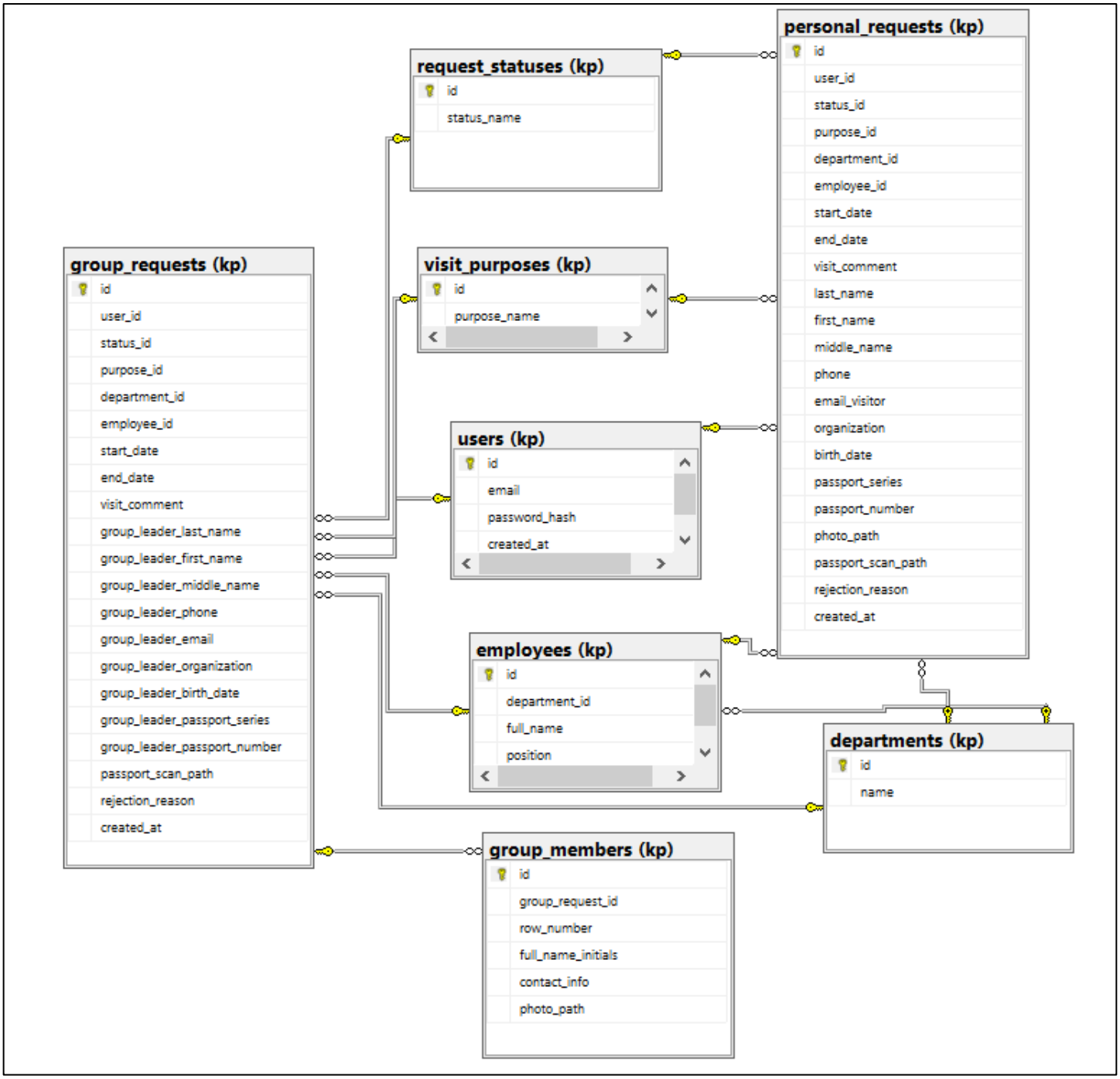

Рисунок 2 - ERD диаграмма

2. ВЫДЕЛЕНИЕ ОБЪЕКТОВ И АТРИБУТОВ В СООТВЕТСТВИИ С ЗАДАНИЕМ

СЛОВАРЬ ДАННЫХ (DATA DICTIONARY)

Таблица users (Пользователи системы)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор пользователя
email	VARCHAR(255)	NOT NULL, UNIQUE	Адрес электронной почты (логин)
login	VARCHAR(255)	UNIQUE	Уникальный логин (формируется автоматически триггером)
password_hash	VARCHAR(255)	NOT NULL	Хеш пароля (MD5)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата и время регистрации

Таблица 1 - users

Таблица request_statuses (Статусы заявок)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор статуса
status_name	VARCHAR(50)	NOT NULL, UNIQUE	Название статуса (проверка, одобрена, не одобрена)

Таблица 2 - request_statuses

Таблица visit_purposes (Цели посещения)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор цели
purpose_name	VARCHAR(255)	NOT NULL, UNIQUE	Название цели посещения

Таблица 3 - visit_purposes

Таблица departments (Подразделения)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор подразделения
name	VARCHAR(255)	NOT NULL, UNIQUE	Название подразделения

Таблица 4 - departments

Таблица employees (Сотрудники)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор сотрудника
department_id	INT	NOT NULL, FOREIGN KEY → departments(id)	ID подразделения
full_name	VARCHAR(255)	NOT NULL	Полное имя сотрудника
position	VARCHAR(255)	NULL	Должность сотрудника

Таблица 5 - employees

Таблица employee_auth (Авторизация сотрудников)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор записи

Поле	Тип данных	Ограничения	Описание
employee_id	INT	NOT NULL, FOREIGN KEY → employees(id)	ID сотрудника
auth_code	VARCHAR(10)	NOT NULL, UNIQUE	Код авторизации
role	VARCHAR(50)	NOT NULL	Роль (GeneralDept, Security, DepartmentEmployee)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата создания

Таблица 6 - employee_auth

Таблица personal_requests (Личные заявки)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор заявки
user_id	INT	NOT NULL, FOREIGN KEY → users(id)	ID пользователя
status_id	INT	NOT NULL, FOREIGN KEY → request_statuses(id)	ID статуса
purpose_id	INT	NOT NULL, FOREIGN KEY → visit_purposes(id)	ID цели
department_id	INT	NOT NULL, FOREIGN KEY → departments(id)	ID подразделения
employee_id	INT	NOT NULL, FOREIGN KEY → employees(id)	ID сотрудника
start_date	DATE	NOT NULL	Дата начала
end_date	DATE	NOT NULL	Дата окончания
visit_comment	TEXT	NOT NULL	Комментарий

Поле	Тип данных	Ограничения	Описание
last_name	VARCHAR(100)	NOT NULL	Фамилия
first_name	VARCHAR(100)	NOT NULL	Имя
middle_name	VARCHAR(100)	NULL	Отчество
phone	VARCHAR(20)	NULL	Телефон
email_visitor	VARCHAR(255)	NOT NULL	Email
organization	VARCHAR(255)	NULL	Организация
birth_date	DATE	NOT NULL	Дата рождения
passport_series	VARCHAR(4)	NOT NULL	Серия паспорта
passport_number	VARCHAR(6)	NOT NULL	Номер паспорта
photo_path	TEXT	NULL	Путь к фото
passport_scan_path	TEXT	NOT NULL	Путь к скану
rejection_reason	TEXT	NULL	Причина отказа
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата создания
visit_start_time	TIMESTAMP	NULL	Время начала посещения
visit_end_time	TIMESTAMP	NULL	Время окончания
blacklist_checked	BOOLEAN	DEFAULT FALSE	Проверка черного списка

Таблица 7 - personal_requests

Таблица blacklist (Черный список)

Поле	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор
passport_series	VARCHAR(4)	NOT NULL	Серия паспорта
passport_number	VARCHAR(6)	NOT NULL	Номер паспорта
last_name	VARCHAR(100)	NOT NULL	Фамилия
first_name	VARCHAR(100)	NOT NULL	Имя
middle_name	VARCHAR(100)	NULL	Отчество
reason	TEXT	NOT NULL	Причина добавления
added_by	INT	FOREIGN KEY → users(id)	Кем добавлен
added_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата добавления

Таблица 8 - blacklist

3. ПРОЕКТИРОВАНИЕ И НОРМАЛИЗАЦИЯ БД

Проектирование базы данных является одним из ключевых этапов разработки информационной системы. На данном этапе была выполнена концептуальная, логическая и физическая модель базы данных системы «ХранительПРО».

Выявление сущностей
По результатам анализа предметной области и построенных UML-диаграмм (см. раздел 1) было выделено **5 основных сущностей**, отражающих все значимые объекты системы контроля физического доступа.

Основные сущности:

- Пользователи (посетители)
- Сотрудники (общего отдела, охраны, подразделений)
- Заявки на посещение (личные и групповые)
- Черный список
- Отчеты

3.1. Нормализация базы данных

База данных спроектирована в соответствии с требованиями нормализации до **3-й нормальной формы (3NF)**:

1-я нормальная форма (1НФ):

- Все таблицы имеют первичный ключ
- Все атрибуты атомарны (не содержат составных значений)
- Нет повторяющихся групп

2-я нормальная форма (2НФ):

- Все неключевые атрибуты зависят от полного первичного ключа
- Устранены частичные зависимости

3-я нормальная форма (3НФ):

- Отсутствуют транзитивные зависимости
- Неключевые атрибуты зависят только от первичного ключа

4. ВЫПОЛНЕНИЕ ПОСТРОЕНИЯ БД В ПРЕДЛОЖЕННОЙ СУБД

4.1. Используемая СУБД

В качестве системы управления базами данных использовалась **PostgreSQL 15**. Выбор обоснован следующими преимуществами:

- Бесплатное и открытое программное обеспечение
- Высокая производительность и надежность
- Поддержка сложных запросов и транзакций
- Возможность создания хранимых процедур и триггеров
- Кроссплатформенность

4.2. SQL скрипт создания базы данных

```sql
-- Создание схемы
CREATE SCHEMA IF NOT EXISTS kp;

-- Таблица статусов заявок
CREATE TABLE IF NOT EXISTS kp.request_statuses (
    id SERIAL PRIMARY KEY,
    status_name VARCHAR(50) NOT NULL UNIQUE
);

-- Таблица целей посещения
CREATE TABLE IF NOT EXISTS kp.visit_purposes (
    id SERIAL PRIMARY KEY,
    purpose_name VARCHAR(255) NOT NULL UNIQUE
);

-- Таблица подразделений
CREATE TABLE IF NOT EXISTS kp.departments (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE
);

-- Таблица сотрудников
CREATE TABLE IF NOT EXISTS kp.employees (
    id SERIAL PRIMARY KEY,
    department_id INT NOT NULL REFERENCES kp.departments(id) ON DELETE CASCADE,
    full_name VARCHAR(255) NOT NULL,
    position VARCHAR(255)
);

-- Таблица пользователей
```

```sql
CREATE TABLE IF NOT EXISTS kp.users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    login VARCHAR(255) UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица личных заявок
CREATE TABLE IF NOT EXISTS kp.personal_requests (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES kp.users(id) ON DELETE CASCADE,
    status_id INT NOT NULL REFERENCES kp.request_statuses(id),
    purpose_id INT NOT NULL REFERENCES kp.visit_purposes(id),
    department_id INT NOT NULL REFERENCES kp.departments(id),
    employee_id INT NOT NULL REFERENCES kp.employees(id),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    visit_comment TEXT NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    phone VARCHAR(20),
    email_visitor VARCHAR(255) NOT NULL,
    organization VARCHAR(255),
    birth_date DATE NOT NULL,
    passport_series VARCHAR(4) NOT NULL,
    passport_number VARCHAR(6) NOT NULL,
    photo_path TEXT,
    passport_scan_path TEXT NOT NULL,
    rejection_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    visit_start_time TIMESTAMP,
    visit_end_time TIMESTAMP,
    blacklist_checked BOOLEAN DEFAULT FALSE
);

-- Индексы
CREATE INDEX idx_personal_user_id ON kp.personal_requests(user_id);
CREATE INDEX idx_personal_status ON kp.personal_requests(status_id);
CREATE INDEX idx_users_email ON kp.users(email);
```

5. РЕАЛИЗАЦИЯ УРОВНЕЙ ДОСТУПА ДЛЯ РАЗЛИЧНЫХ КАТЕГОРИЙ ПОЛЬЗОВАТЕЛЕЙ

5.1. Роли пользователей

Роль	Описание	Доступные функции
Посетитель	Незарегистрированный пользователь	Регистрация, подача заявки
Авторизованный пользователь	Зарегистрированный посетитель	Авторизация, подача заявок, просмотр своих заявок
GeneralDept	Сотрудник общего отдела	Просмотр всех заявок, фильтрация, одобрение/отклонение, формирование отчетов
Security	Сотрудник охраны	Просмотр одобренных заявок, разрешение доступа
DepartmentEmployee	Сотрудник подразделения	Просмотр заявок своего отдела, фиксация времени, добавление в черный список

Таблица 9 – Роли пользователей

5.2. Таблица employee_auth

```sql
CREATE TABLE kp.employee_auth (
    id SERIAL PRIMARY KEY,
    employee_id INT NOT NULL REFERENCES kp.employees(id),
    auth_code VARCHAR(10) NOT NULL UNIQUE,
    role VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Тестовые данные
INSERT INTO kp.employee_auth (employee_id, auth_code, role) VALUES
(1, 'GEN001', 'GeneralDept'),
(2, 'GEN002', 'GeneralDept'),
(3, 'SEC001', 'Security'),
(4, 'DEP001', 'DepartmentEmployee');
```

5.3. Функция авторизации сотрудника

```sql
CREATE OR REPLACE FUNCTION kp.auth_employee(p_auth_code VARCHAR)
RETURNS TABLE(
    employee_id INT,
    full_name VARCHAR,
    role VARCHAR,
    department_name VARCHAR
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        e.id,
        e.full_name,
        ea.role,
        d.name
    FROM kp.employee_auth ea
    JOIN kp.employees e ON e.id = ea.employee_id
    JOIN kp.departments d ON d.id = e.department_id
    WHERE ea.auth_code = p_auth_code;
END;
$$ LANGUAGE plpgsql;
```

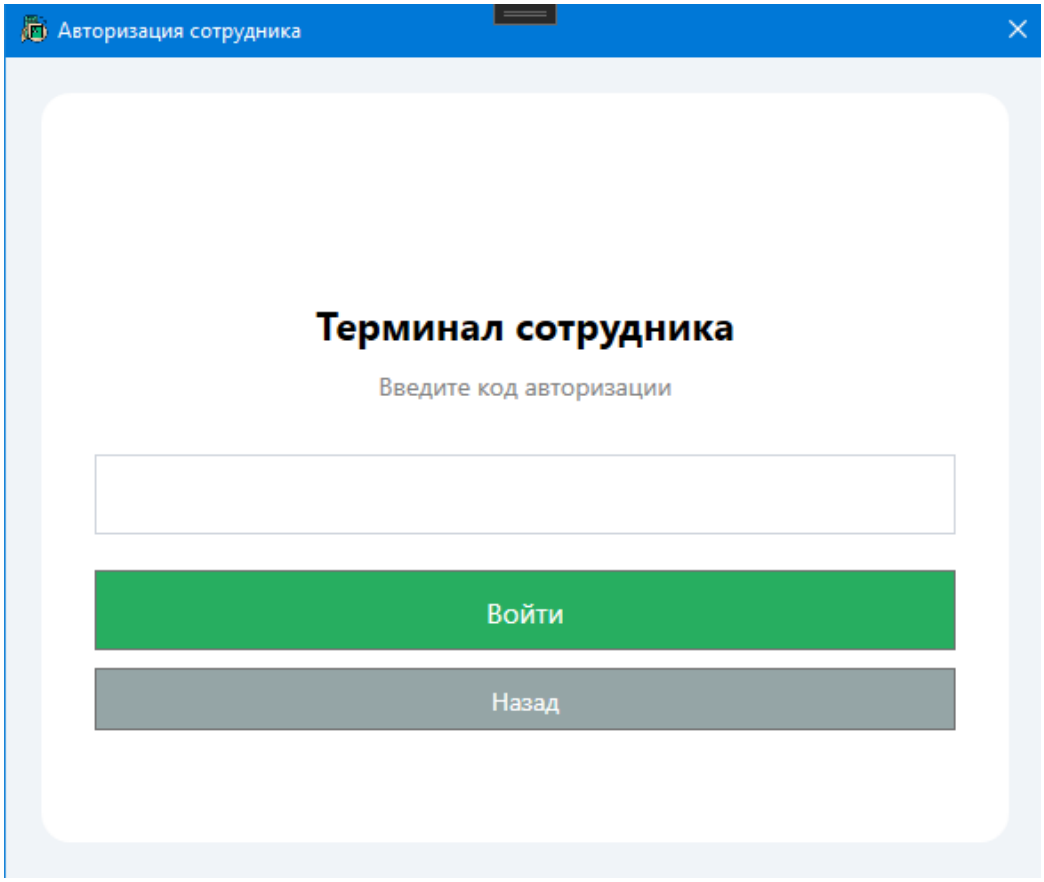

Рисунок 3 – Окно авторизации сотрудника

6. СОЗДАНИЕ ЗАПРОСОВ И ОТЧЕТОВ В СООТВЕТСТВИИ С ЗАДАНИЕМ

6.1. Представление для просмотра всех заявок

```sql
CREATE OR REPLACE VIEW kp.view_list_requests AS
SELECT
    pr.id AS request_id,
    'Личная' AS request_type,
    u.email AS user_email,
    d.name AS department_name,
    e.full_name AS employee_name,
    pr.start_date,
    pr.end_date,
    pr.last_name,
    pr.first_name,
    pr.middle_name,
    pr.phone,
    pr.email_visitor,
    pr.passport_series,
    pr.passport_number,
    rs.status_name AS status,
    pr.rejection_reason,
    pr.created_at,
    pr.visit_start_time,
    pr.visit_end_time
FROM kp.personal_requests pr
JOIN kp.users u ON u.id = pr.user_id
JOIN kp.departments d ON d.id = pr.department_id
JOIN kp.employees e ON e.id = pr.employee_id
JOIN kp.request_statuses rs ON rs.id = pr.status_id

UNION ALL

SELECT
    gr.id,
    'Групповая',
    u.email,
    d.name,
    e.full_name,
    gr.start_date,
    gr.end_date,
    gr.group_leader_last_name,
    gr.group_leader_first_name,
    gr.group_leader_middle_name,
    gr.group_leader_phone,
```

```
    gr.group_leader_email,
    gr.group_leader_passport_series,
    gr.group_leader_passport_number,
    rs.status_name,
    gr.rejection_reason,
    gr.created_at,
    gr.visit_start_time,
    gr.visit_end_time
FROM kp.group_requests gr
JOIN kp.users u ON u.id = gr.user_id
JOIN kp.departments d ON d.id = gr.department_id
JOIN kp.employees e ON e.id = gr.employee_id
JOIN kp.request_statuses rs ON rs.id = gr.status_id;
```

6.2. Хранимая процедура фильтрации заявок

```
sql
CREATE OR REPLACE FUNCTION kp.filtering_requests(
    p_request_type VARCHAR DEFAULT NULL,
    p_department_id INT DEFAULT NULL,
    p_status_id INT DEFAULT NULL,
    p_date_from DATE DEFAULT NULL,
    p_date_to DATE DEFAULT NULL,
    p_visitor_name VARCHAR DEFAULT NULL
)
RETURNS TABLE(
    request_id INT,
    request_type VARCHAR,
    user_email VARCHAR,
    department_name VARCHAR,
    employee_name VARCHAR,
    start_date DATE,
    end_date DATE,
    last_name VARCHAR,
    first_name VARCHAR,
    status VARCHAR,
    created_at TIMESTAMP
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        v.request_id,
        v.request_type,
        v.user_email,
        v.department_name,
        v.employee_name,
        v.start_date,
```

```
        v.end_date,
        v.last_name,
        v.first_name,
        v.status,
        v.created_at
    FROM kp.view_list_requests v
    WHERE (p_request_type IS NULL OR v.request_type = p_request_type)
        AND (p_department_id IS NULL OR v.department_id = p_department_id)
        AND (p_status_id IS NULL OR v.status_id = p_status_id)
        AND (p_date_from IS NULL OR DATE(v.created_at) >= p_date_from)
        AND (p_date_to IS NULL OR DATE(v.created_at) <= p_date_to)
        AND (p_visitor_name IS NULL OR
            v.last_name ILIKE '%' || p_visitor_name || '%' OR
            v.first_name ILIKE '%' || p_visitor_name || '%')
    ORDER BY v.created_at DESC;
END;
$$ LANGUAGE plpgsql;
```

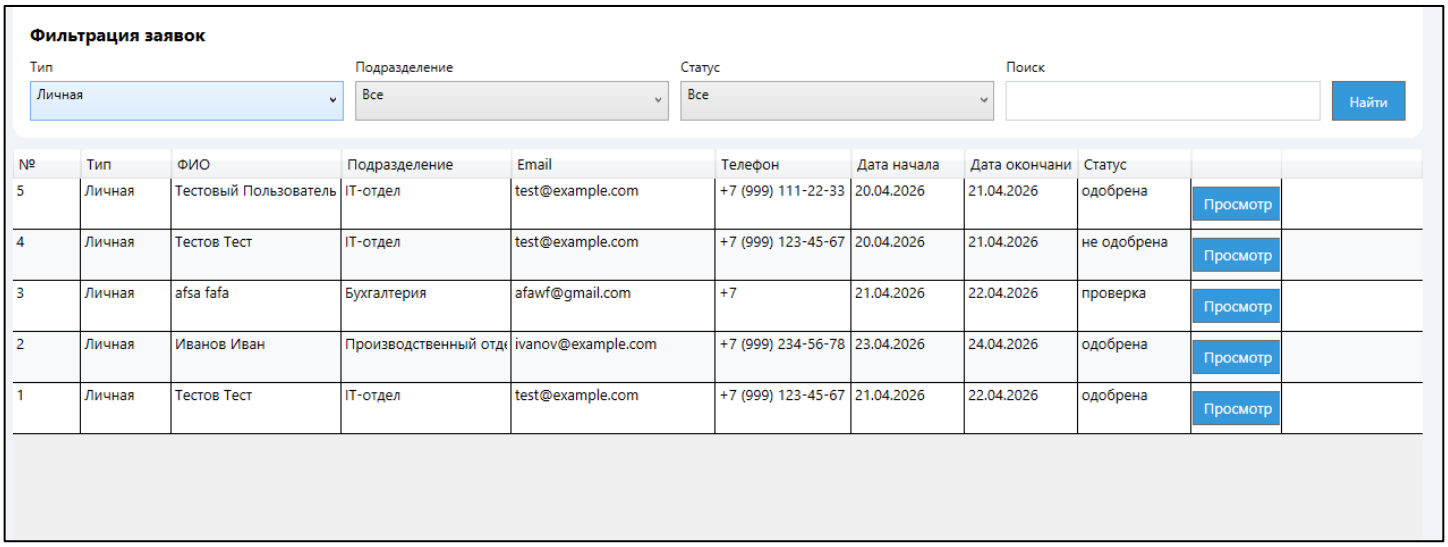

Рисунок 4 – Фильтрация заявок

6.3. Триггер для автоматического формирования логина

```
sql
CREATE OR REPLACE FUNCTION kp.generate_visitor_login()
RETURNS TRIGGER AS $$
DECLARE
    login_prefix VARCHAR;
    at_position INT;
    new_login VARCHAR;
BEGIN
    at_position := POSITION('@' IN NEW.email);
    IF at_position > 0 THEN
```

```
        login_prefix := LEFT(NEW.email, at_position - 1);
    ELSE
        login_prefix := NEW.email;
    END IF;

    new_login := login_prefix || '_' || NEW.id;

    UPDATE kp.users SET login = new_login WHERE id = NEW.id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_generate_visitor_login
    AFTER INSERT ON kp.users
    FOR EACH ROW
    EXECUTE FUNCTION kp.generate_visitor_login();
```

6.4. Триггер для автоматического добавления в черный список

```sql
CREATE OR REPLACE FUNCTION kp.auto_add_to_blacklist()
RETURNS TRIGGER AS $$
DECLARE
    reject_count INT;
BEGIN
    IF NEW.status_id = 3 AND NEW.rejection_reason LIKE '%недостоверных данных%' THEN
        SELECT COUNT(*) INTO reject_count
        FROM kp.personal_requests
        WHERE passport_series = NEW.passport_series
          AND passport_number = NEW.passport_number
          AND status_id = 3;
        IF reject_count >= 2 THEN
            INSERT INTO kp.blacklist (passport_series, passport_number, last_name, first_name, middle_name, reason)
            VALUES (NEW.passport_series, NEW.passport_number, NEW.last_name, NEW.first_name, NEW.middle_name,
                    'Автоматическое добавление: двукратное указание недостоверных данных');
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_auto_blacklist_personal
    AFTER UPDATE OF status_id ON kp.personal_requests
    FOR EACH ROW
    WHEN (OLD.status_id != 3 AND NEW.status_id = 3)
    EXECUTE FUNCTION kp.auto_add_to_blacklist();
```

7. СОЗДАНИЕ И ОБОСНОВАНИЕ ГРУПП ПОЛЬЗОВАТЕЛЕЙ, ПРИНЦИПЫ РЕГИСТРАЦИИ И СИСТЕМЫ ПАРОЛЕЙ

7.1. Группы пользователей

Группа	Способ входа	Уровень доступа
Посетители	Email + пароль	Только подача заявок и просмотр своих заявок
Сотрудники	Код авторизации	Расширенный доступ в зависимости от роли

7.2. Принципы регистрации

- Регистрация по email и паролю
- Валидация email (проверка формата)
- Требования к паролю: минимум 8 символов, заглавные и строчные буквы, цифры, спецсимволы
- Хранение паролей в зашифрованном виде (MD5)

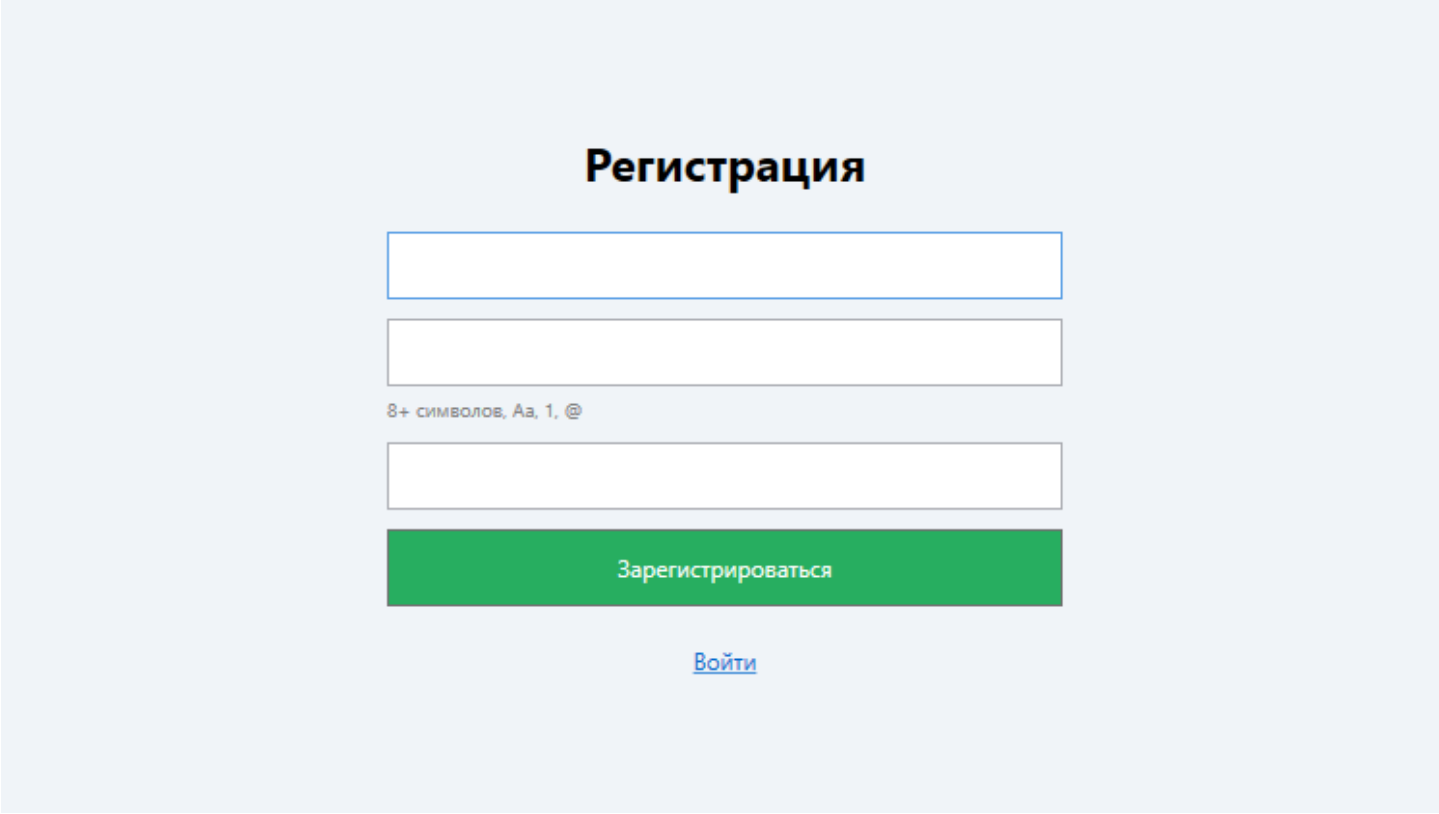

Рисунок 4 – Регистрация

7.3. Система паролей

```csharp
private string GetMD5Hash(string input)
{
    using (MD5 md5 = MD5.Create())
    {
        byte[] inputBytes = Encoding.UTF8.GetBytes(input);
        byte[] hashBytes = md5.ComputeHash(inputBytes);
        StringBuilder sb = new StringBuilder();
        foreach (byte b in hashBytes)
            sb.Append(b.ToString("x2"));
        return sb.ToString();
    }
}

private bool IsValidPassword(string password)
{
    if (password.Length < 8) return false;
    if (!Regex.IsMatch(password, @"[A-Z]")) return false;
    if (!Regex.IsMatch(password, @"[a-z]")) return false;
    if (!Regex.IsMatch(password, @"[0-9]")) return false;
    if (!Regex.IsMatch(password, @"[!@#$%^&*()]")) return false;
    return true;
}
```

7.4. Тестовые данные для входа

Роль	Логин/Код	Пароль
Посетитель	test@example.com	Test123!
Сотрудник ОО	GEN001	-
Сотрудник ОО	GEN002	-
Сотрудник охраны	SEC001	-
Сотрудник подразделения	DEP001	-

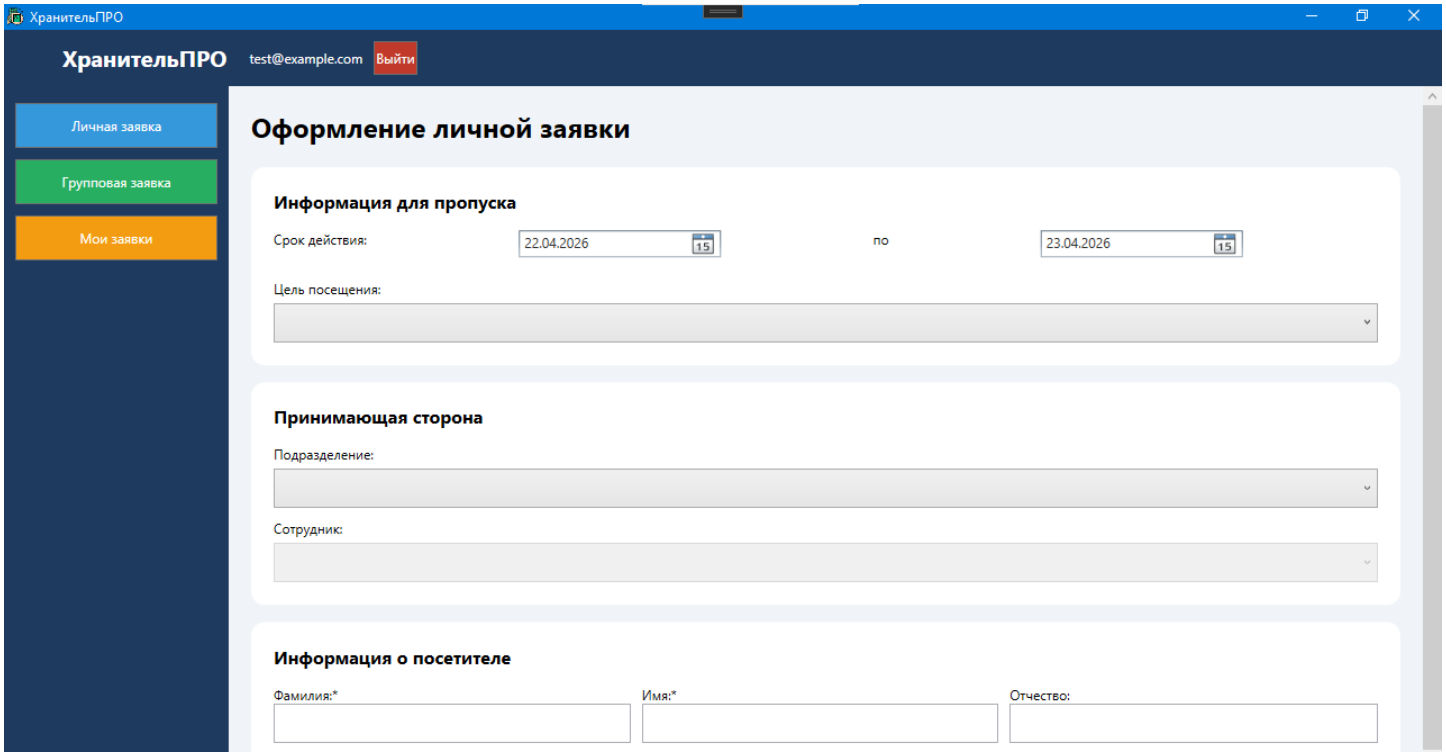

Рисунок 5 – Вход под тестовым посетителем

8. ВЫПОЛНЕНИЕ РЕЗЕРВНОГО КОПИРОВАНИЯ БД И ВОССТАНОВЛЕНИЕ СОСТОЯНИЯ БД

8.1. Создание резервной копии (pg_dump)

bash

```
# Создание резервной копии всей базы данных
pg_dump -U postgres -h localhost -F c -b -v -f "KhranitelPro_backup.dump" KhranitelPro

# Создание резервной копии в формате SQL
pg_dump -U postgres -h localhost -f "KhranitelPro_backup.sql" KhranitelPro
```

8.2. Восстановление из резервной копии

bash

```
# Восстановление из дампа
pg_restore -U postgres -h localhost -d KhranitelPro -v "KhranitelPro_backup.dump"

# Восстановление из SQL файла
psql -U postgres -h localhost -d KhranitelPro -f "KhranitelPro_backup.sql"
```

8.3. Скрипт автоматического резервного копирования

sql

```
-- Создание резервной копии через SQL
```

```
COPY (SELECT * FROM kp.personal_requests) TO '/backup/personal_requests.csv' CSV HEADER;
COPY (SELECT * FROM kp.users) TO '/backup/users.csv' CSV HEADER;
```

9. ЗАКЛЮЧЕНИЕ

В ходе выполнения практики были решены следующие задачи:

1. **Разработана база данных** для системы контроля доступа "ХранительПРО" в СУБД PostgreSQL
2. **Созданы три настольных приложения**:
   - Приложение для посетителей (подача заявок)
   - Терминал сотрудника общего отдела (проверка заявок, отчеты)
   - Терминал сотрудника охраны (разрешение доступа)
   - Терминал сотрудника подразделения (фиксация времени, черный список)
3. **Реализованы уровни доступа** для различных категорий пользователей
4. **Созданы хранимые процедуры и триггеры** для автоматизации процессов
5. **Разработаны отчеты** о посещениях и текущих посетителях
6. **Настроено автоматическое формирование отчетов** по требованию ТБ

Результаты работы:

- Полностью функционирующая информационная система
- Интуитивно понятный интерфейс
- Высокий уровень безопасности (MD5 хеширование, ролевая модель)
- Автоматизация ключевых бизнес-процессов

Приложения

1. Скрипт создания базы данных

```sql
-- ==========================================================
-- ПОЛНЫЙ SQL СКРИПТ ДЛЯ БАЗЫ ДАННЫХ "ХранительПРО"
-- PostgreSQL
-- Версия: 1.0
-- Дата: 2024
-- ==========================================================

-- ==========================================================
-- 1. СОЗДАНИЕ СХЕМЫ
-- ==========================================================

DROP SCHEMA IF EXISTS kp CASCADE;
CREATE SCHEMA kp;

SET search_path TO kp;

-- ==========================================================
-- 2. ОСНОВНЫЕ ТАБЛИЦЫ
-- ==========================================================

-- Таблица статусов заявок
CREATE TABLE kp.request_statuses (
    id SERIAL PRIMARY KEY,
    status_name VARCHAR(50) NOT NULL UNIQUE
);

-- Таблица целей посещения
CREATE TABLE kp.visit_purposes (
    id SERIAL PRIMARY KEY,
    purpose_name VARCHAR(255) NOT NULL UNIQUE
);

-- Таблица подразделений
CREATE TABLE kp.departments (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE
);

-- Таблица сотрудников
CREATE TABLE kp.employees (
    id SERIAL PRIMARY KEY,
    department_id INT NOT NULL,
```

```sql
    full_name VARCHAR(255) NOT NULL,
    position VARCHAR(255),
    CONSTRAINT fk_employees_department FOREIGN KEY (department_id) REFERENCES kp.departments(id) ON DELETE CASCADE
);

-- Таблица пользователей (посетители)
CREATE TABLE kp.users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    login VARCHAR(255) UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица личных заявок
CREATE TABLE kp.personal_requests (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    status_id INT NOT NULL,
    purpose_id INT NOT NULL,
    department_id INT NOT NULL,
    employee_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    visit_comment TEXT NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    phone VARCHAR(20),
    email_visitor VARCHAR(255) NOT NULL,
    organization VARCHAR(255),
    birth_date DATE NOT NULL,
    passport_series VARCHAR(4) NOT NULL,
    passport_number VARCHAR(6) NOT NULL,
    photo_path TEXT,
    passport_scan_path TEXT NOT NULL,
    rejection_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    visit_start_time TIMESTAMP,
    visit_end_time TIMESTAMP,
    blacklist_checked BOOLEAN DEFAULT FALSE,
    CONSTRAINT fk_personal_user FOREIGN KEY (user_id) REFERENCES kp.users(id) ON DELETE CASCADE,
    CONSTRAINT fk_personal_status FOREIGN KEY (status_id) REFERENCES kp.request_statuses(id),
    CONSTRAINT fk_personal_purpose FOREIGN KEY (purpose_id) REFERENCES kp.visit_purposes(id),
    CONSTRAINT fk_personal_department FOREIGN KEY (department_id) REFERENCES kp.departments(id),
    CONSTRAINT fk_personal_employee FOREIGN KEY (employee_id) REFERENCES kp.employees(id)
);
```

```sql
-- Таблица групповых заявок
CREATE TABLE kp.group_requests (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    status_id INT NOT NULL,
    purpose_id INT NOT NULL,
    department_id INT NOT NULL,
    employee_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    visit_comment TEXT NOT NULL,
    group_leader_last_name VARCHAR(100) NOT NULL,
    group_leader_first_name VARCHAR(100) NOT NULL,
    group_leader_middle_name VARCHAR(100),
    group_leader_phone VARCHAR(20),
    group_leader_email VARCHAR(255) NOT NULL,
    group_leader_organization VARCHAR(255),
    group_leader_birth_date DATE NOT NULL,
    group_leader_passport_series VARCHAR(4) NOT NULL,
    group_leader_passport_number VARCHAR(6) NOT NULL,
    passport_scan_path TEXT NOT NULL,
    rejection_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    visit_start_time TIMESTAMP,
    visit_end_time TIMESTAMP,
    blacklist_checked BOOLEAN DEFAULT FALSE,
    CONSTRAINT fk_group_user FOREIGN KEY (user_id) REFERENCES kp.users(id) ON DELETE CASCADE,
    CONSTRAINT fk_group_status FOREIGN KEY (status_id) REFERENCES kp.request_statuses(id),
    CONSTRAINT fk_group_purpose FOREIGN KEY (purpose_id) REFERENCES kp.visit_purposes(id),
    CONSTRAINT fk_group_department FOREIGN KEY (department_id) REFERENCES kp.departments(id),
    CONSTRAINT fk_group_employee FOREIGN KEY (employee_id) REFERENCES kp.employees(id)
);

-- Таблица членов группы
CREATE TABLE kp.group_members (
    id SERIAL PRIMARY KEY,
    group_request_id INT NOT NULL,
    row_number INT NOT NULL,
    full_name_initials VARCHAR(255) NOT NULL,
    contact_info TEXT NOT NULL,
    photo_path TEXT,
    CONSTRAINT fk_members_group FOREIGN KEY (group_request_id) REFERENCES kp.group_requests(id) ON DELETE CASCADE
);

-- ======================================================
-- 3. ТАБЛИЦЫ ДЛЯ АВТОРИЗАЦИИ И БЕЗОПАСНОСТИ
```

```sql
-- ==========================================================

-- Таблица авторизации сотрудников
CREATE TABLE kp.employee_auth (
    id SERIAL PRIMARY KEY,
    employee_id INT NOT NULL,
    auth_code VARCHAR(10) NOT NULL UNIQUE,
    role VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_employee_auth_employees FOREIGN KEY (employee_id) REFERENCES kp.employees(id) ON DELETE CASCADE
);

-- Таблица черного списка
CREATE TABLE kp.blacklist (
    id SERIAL PRIMARY KEY,
    passport_series VARCHAR(4) NOT NULL,
    passport_number VARCHAR(6) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    reason TEXT NOT NULL,
    added_by INT,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_blacklist_users FOREIGN KEY (added_by) REFERENCES kp.users(id) ON DELETE SET NULL
);

-- Таблица уведомлений
CREATE TABLE kp.notifications (
    id SERIAL PRIMARY KEY,
    user_id INT,
    request_id INT,
    request_type VARCHAR(50),
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES kp.users(id) ON DELETE CASCADE
);

-- Таблица отчетов ТБ
CREATE TABLE kp.safety_reports (
    id SERIAL PRIMARY KEY,
    report_date DATE NOT NULL,
    report_time TIME NOT NULL,
    department_id INT,
    visitors_count INT NOT NULL,
    report_path TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```sql
    CONSTRAINT fk_report_department FOREIGN KEY (department_id) REFERENCES kp.departments(id)
);

-- Таблица конфигурации времени
CREATE TABLE kp.time_config (
    id SERIAL PRIMARY KEY,
    from_gate_to_department_minutes INT NOT NULL DEFAULT 15,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_by INT,
    CONSTRAINT fk_time_config_user FOREIGN KEY (updated_by) REFERENCES kp.employees(id)
);

-- Таблица нарушений
CREATE TABLE kp.violations (
    id SERIAL PRIMARY KEY,
    request_id INT,
    request_type VARCHAR(50),
    visitor_name VARCHAR(255) NOT NULL,
    violation_type VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ====================================================
-- 4. ИНДЕКСЫ
-- ====================================================

CREATE INDEX idx_blacklist_passport ON kp.blacklist(passport_series, passport_number);
CREATE INDEX idx_personal_user_id ON kp.personal_requests(user_id);
CREATE INDEX idx_personal_status ON kp.personal_requests(status_id);
CREATE INDEX idx_personal_dates ON kp.personal_requests(start_date, end_date);
CREATE INDEX idx_group_user_id ON kp.group_requests(user_id);
CREATE INDEX idx_group_status ON kp.group_requests(status_id);
CREATE INDEX idx_group_dates ON kp.group_requests(start_date, end_date);
CREATE INDEX idx_group_members_request ON kp.group_members(group_request_id);
CREATE INDEX idx_users_email ON kp.users(email);
CREATE INDEX idx_users_login ON kp.users(login);
CREATE INDEX idx_employee_auth_code ON kp.employee_auth(auth_code);
CREATE INDEX idx_notifications_user ON kp.notifications(user_id);
CREATE INDEX idx_notifications_is_read ON kp.notifications(is_read);
CREATE INDEX idx_safety_reports_date ON kp.safety_reports(report_date);
CREATE INDEX idx_violations_created ON kp.violations(created_at);

-- ====================================================
-- 5. ЗАПОЛНЕНИЕ СПРАВОЧНИКОВ
-- ====================================================
```

```sql
INSERT INTO kp.request_statuses (status_name) VALUES
('проверка'), ('одобрена'), ('не одобрена')
ON CONFLICT DO NOTHING;

INSERT INTO kp.visit_purposes (purpose_name) VALUES
('Рабочая встреча'), ('Экскурсия'), ('Техническое обслуживание')
ON CONFLICT DO NOTHING;

INSERT INTO kp.departments (name) VALUES
('IT-отдел'),
('Отдел безопасности'),
('Бухгалтерия'),
('Отдел кадров'),
('Производственный отдел'),
('Отдел разработки'),
('Технический отдел')
ON CONFLICT DO NOTHING;

-- ======================================================
-- 6. СОТРУДНИКИ
-- ======================================================

INSERT INTO kp.employees (department_id, full_name, position) VALUES
(1, 'Иванов Иван Иванович', 'Начальник IT'),
(1, 'Петров Петр Петрович', 'Инженер'),
(2, 'Сидоров Сидор Сидорович', 'Начальник охраны'),
(3, 'Кузнецова Мария Ивановна', 'Главный бухгалтер'),
(4, 'Петрова Анна Сергеевна', 'Специалист общего отдела'),
(4, 'Смирнов Дмитрий Алексеевич', 'Старший специалист общего отдела'),
(2, 'Орлов Иван Петрович', 'Начальник охраны'),
(5, 'Михайлов Михаил Михайлович', 'Начальник производства'),
(6, 'Соколова Елена Владимировна', 'Руководитель отдела'),
(7, 'Васильев Андрей Николаевич', 'Главный инженер')
ON CONFLICT DO NOTHING;

-- ======================================================
-- 7. КОДЫ АВТОРИЗАЦИИ ДЛЯ СОТРУДНИКОВ
-- ======================================================

INSERT INTO kp.employee_auth (employee_id, auth_code, role) VALUES
((SELECT id FROM kp.employees WHERE full_name = 'Петрова Анна Сергеевна'), 'GEN001', 'GeneralDept'),
((SELECT id FROM kp.employees WHERE full_name = 'Смирнов Дмитрий Алексеевич'), 'GEN002', 'GeneralDept'),
((SELECT id FROM kp.employees WHERE full_name = 'Орлов Иван Петрович'), 'SEC001', 'Security'),
((SELECT id FROM kp.employees WHERE full_name = 'Михайлов Михаил Михайлович'), 'DEP001', 'DepartmentEmployee'),
((SELECT id FROM kp.employees WHERE full_name = 'Соколова Елена Владимировна'), 'DEP002', 'DepartmentEmployee')
ON CONFLICT (auth_code) DO NOTHING;
```

```sql
-- ========================================================
-- 8. ТЕСТОВЫЕ ПОЛЬЗОВАТЕЛИ
-- ========================================================

INSERT INTO kp.users (email, password_hash, created_at) VALUES
('test@example.com', MD5('Test123!'), NOW()),
('ivanov@example.com', MD5('User123!'), NOW()),
('petrov@example.com', MD5('Pass123!'), NOW())
ON CONFLICT (email) DO NOTHING;

-- ========================================================
-- 9. ТЕСТОВЫЙ ЧЕРНЫЙ СПИСОК
-- ========================================================

INSERT INTO kp.blacklist (passport_series, passport_number, last_name, first_name, middle_name, reason) VALUES
('1234', '567890', 'Иванов', 'Петр', 'Сидорович', 'Нарушение правил посещения: пронос запрещенных предметов'),
('1111', '222222', 'Сидоров', 'Алексей', 'Иванович', 'Неоднократное нарушение пропускного режима')
ON CONFLICT DO NOTHING;

-- ========================================================
-- 10. КОНФИГУРАЦИЯ ВРЕМЕНИ
-- ========================================================

INSERT INTO kp.time_config (from_gate_to_department_minutes) VALUES (15)
ON CONFLICT DO NOTHING;

-- ========================================================
-- 11. ТЕСТОВЫЕ ЗАЯВКИ
-- ========================================================

-- Личная заявка (статус: проверка)
INSERT INTO kp.personal_requests (
    user_id, status_id, purpose_id, department_id, employee_id,
    start_date, end_date, visit_comment,
    last_name, first_name, middle_name, phone, email_visitor,
    birth_date, passport_series, passport_number, passport_scan_path, created_at
) VALUES (
    (SELECT id FROM kp.users WHERE email = 'test@example.com'),
    (SELECT id FROM kp.request_statuses WHERE status_name = 'проверка'),
    (SELECT id FROM kp.visit_purposes WHERE purpose_name = 'Рабочая встреча'),
    (SELECT id FROM kp.departments WHERE name = 'IT-отдел'),
    (SELECT id FROM kp.employees WHERE full_name = 'Иванов Иван Иванович'),
    CURRENT_DATE + 1, CURRENT_DATE + 2,
    'Обсуждение технического задания',
    'Тестов', 'Тест', 'Тестович',
    '+7 (999) 123-45-67', 'test@example.com',
    '1990-01-01', '1111', '111111',
```

```
    '/scans/test_passport.pdf',
    NOW()
);

-- Личная заявка (статус: одобрена)
INSERT INTO kp.personal_requests (
    user_id, status_id, purpose_id, department_id, employee_id,
    start_date, end_date, visit_comment,
    last_name, first_name, middle_name, phone, email_visitor,
    birth_date, passport_series, passport_number, passport_scan_path, created_at
) VALUES (
    (SELECT id FROM kp.users WHERE email = 'ivanov@example.com'),
    (SELECT id FROM kp.request_statuses WHERE status_name = 'одобрена'),
    (SELECT id FROM kp.visit_purposes WHERE purpose_name = 'Техническое обслуживание'),
    (SELECT id FROM kp.departments WHERE name = 'Производственный отдел'),
    (SELECT id FROM kp.employees WHERE full_name = 'Михайлов Михаил Михайлович'),
    CURRENT_DATE + 3, CURRENT_DATE + 4,
    'Плановое обслуживание оборудования',
    'Иванов', 'Иван', 'Иванович',
    '+7 (999) 234-56-78', 'ivanov@example.com',
    '1985-05-15', '2222', '333333',
    '/scans/ivanov_passport.pdf',
    NOW()
);

-- ======================================================
-- 12. ПРЕДСТАВЛЕНИЯ
-- ======================================================

-- Представление для просмотра всех заявок
DROP VIEW IF EXISTS kp.view_list_requests CASCADE;
CREATE VIEW kp.view_list_requests AS
SELECT
    pr.id AS request_id,
    'Личная' AS request_type,
    u.id AS user_id,
    u.email AS user_email,
    d.id AS department_id,
    d.name AS department_name,
    e.id AS employee_id,
    e.full_name AS employee_name,
    pr.start_date,
    pr.end_date,
    pr.visit_comment,
    pr.last_name,
    pr.first_name,
    pr.middle_name,
```

```
    pr.phone,
    pr.email_visitor AS visitor_email,
    pr.organization,
    pr.birth_date,
    pr.passport_series,
    pr.passport_number,
    pr.photo_path,
    pr.passport_scan_path,
    rs.id AS status_id,
    rs.status_name AS status,
    pr.rejection_reason,
    pr.created_at,
    pr.visit_start_time,
    pr.visit_end_time,
    NULL::VARCHAR AS group_leader_name,
    NULL::BIGINT AS members_count,
    vp.purpose_name
FROM kp.personal_requests pr
INNER JOIN kp.users u ON u.id = pr.user_id
INNER JOIN kp.departments d ON d.id = pr.department_id
INNER JOIN kp.employees e ON e.id = pr.employee_id
INNER JOIN kp.request_statuses rs ON rs.id = pr.status_id
INNER JOIN kp.visit_purposes vp ON vp.id = pr.purpose_id

UNION ALL

SELECT
    gr.id AS request_id,
    'Групповая' AS request_type,
    u.id AS user_id,
    u.email AS user_email,
    d.id AS department_id,
    d.name AS department_name,
    e.id AS employee_id,
    e.full_name AS employee_name,
    gr.start_date,
    gr.end_date,
    gr.visit_comment,
    gr.group_leader_last_name AS last_name,
    gr.group_leader_first_name AS first_name,
    gr.group_leader_middle_name AS middle_name,
    gr.group_leader_phone AS phone,
    gr.group_leader_email AS visitor_email,
    gr.group_leader_organization AS organization,
    gr.group_leader_birth_date AS birth_date,
    gr.group_leader_passport_series AS passport_series,
    gr.group_leader_passport_number AS passport_number,
```

```
    NULL::TEXT AS photo_path,
    gr.passport_scan_path,
    rs.id AS status_id,
    rs.status_name AS status,
    gr.rejection_reason,
    gr.created_at,
    gr.visit_start_time,
    gr.visit_end_time,
    gr.group_leader_last_name || ' ' || gr.group_leader_first_name AS group_leader_name,
    (SELECT COUNT(*)::BIGINT FROM kp.group_members WHERE group_request_id = gr.id) AS members_count,
    vp.purpose_name
FROM kp.group_requests gr
INNER JOIN kp.users u ON u.id = gr.user_id
INNER JOIN kp.departments d ON d.id = gr.department_id
INNER JOIN kp.employees e ON e.id = gr.employee_id
INNER JOIN kp.request_statuses rs ON rs.id = gr.status_id
INNER JOIN kp.visit_purposes vp ON vp.id = gr.purpose_id;

-- ======================================================
-- 13. ХРАНИМЫЕ ПРОЦЕДУРЫ И ФУНКЦИИ
-- ======================================================

-- Функция фильтрации заявок
CREATE OR REPLACE FUNCTION kp.filtering_requests(
    p_request_type VARCHAR DEFAULT NULL,
    p_department_id INT DEFAULT NULL,
    p_status_id INT DEFAULT NULL,
    p_date_from DATE DEFAULT NULL,
    p_date_to DATE DEFAULT NULL,
    p_visitor_name VARCHAR DEFAULT NULL
)
RETURNS TABLE(
    request_id INT,
    request_type VARCHAR,
    user_email VARCHAR,
    department_name VARCHAR,
    employee_name VARCHAR,
    start_date DATE,
    end_date DATE,
    visit_comment TEXT,
    last_name VARCHAR,
    first_name VARCHAR,
    middle_name VARCHAR,
    phone VARCHAR,
    visitor_email VARCHAR,
    organization VARCHAR,
    birth_date DATE,
```

```
    passport_series VARCHAR,
    passport_number VARCHAR,
    status VARCHAR,
    rejection_reason TEXT,
    created_at TIMESTAMP,
    group_leader_name VARCHAR,
    members_count BIGINT,
    visit_start_time TIMESTAMP,
    visit_end_time TIMESTAMP
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        v.request_id,
        v.request_type,
        v.user_email,
        v.department_name,
        v.employee_name,
        v.start_date,
        v.end_date,
        v.visit_comment,
        v.last_name,
        v.first_name,
        v.middle_name,
        v.phone,
        v.visitor_email,
        v.organization,
        v.birth_date,
        v.passport_series,
        v.passport_number,
        v.status,
        v.rejection_reason,
        v.created_at,
        v.group_leader_name,
        v.members_count,
        v.visit_start_time,
        v.visit_end_time
    FROM kp.view_list_requests v
    WHERE (p_request_type IS NULL OR v.request_type = p_request_type)
        AND (p_department_id IS NULL OR v.department_id = p_department_id)
        AND (p_status_id IS NULL OR v.status_id = p_status_id)
        AND (p_date_from IS NULL OR DATE(v.created_at) >= p_date_from)
        AND (p_date_to IS NULL OR DATE(v.created_at) <= p_date_to)
        AND (p_visitor_name IS NULL OR
            v.last_name ILIKE '%' || p_visitor_name || '%' OR
            v.first_name ILIKE '%' || p_visitor_name || '%' OR
            v.middle_name ILIKE '%' || p_visitor_name || '%')
```

```
    ORDER BY v.created_at DESC;
END;
$$ LANGUAGE plpgsql;

-- Функция авторизации сотрудника
CREATE OR REPLACE FUNCTION kp.auth_employee(p_auth_code VARCHAR)
RETURNS TABLE(
    employee_id INT,
    full_name VARCHAR,
    role VARCHAR,
    department_name VARCHAR
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        e.id,
        e.full_name,
        ea.role,
        d.name
    FROM kp.employee_auth ea
    INNER JOIN kp.employees e ON e.id = ea.employee_id
    INNER JOIN kp.departments d ON d.id = e.department_id
    WHERE ea.auth_code = p_auth_code;
END;
$$ LANGUAGE plpgsql;

-- Функция проверки черного списка
CREATE OR REPLACE FUNCTION kp.check_blacklist(
    p_passport_series VARCHAR,
    p_passport_number VARCHAR
)
RETURNS TABLE(
    is_in_blacklist BOOLEAN,
    reason TEXT
) AS $$
BEGIN
    RETURN QUERY
    SELECT TRUE, b.reason
    FROM kp.blacklist b
    WHERE b.passport_series = p_passport_series
      AND b.passport_number = p_passport_number;

    IF NOT FOUND THEN
        RETURN QUERY SELECT FALSE, NULL::TEXT;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
-- Функция обновления статуса заявки
CREATE OR REPLACE FUNCTION kp.update_request_status(
    p_request_id INT,
    p_request_type VARCHAR,
    p_new_status_id INT,
    p_rejection_reason TEXT DEFAULT NULL,
    p_visit_start_time TIMESTAMP DEFAULT NULL
)
RETURNS BOOLEAN AS $$
BEGIN
    IF p_request_type = 'Личная' THEN
        UPDATE kp.personal_requests
        SET status_id = p_new_status_id,
            rejection_reason = COALESCE(p_rejection_reason, rejection_reason),
            visit_start_time = COALESCE(p_visit_start_time, visit_start_time)
        WHERE id = p_request_id;
    ELSE
        UPDATE kp.group_requests
        SET status_id = p_new_status_id,
            rejection_reason = COALESCE(p_rejection_reason, rejection_reason),
            visit_start_time = COALESCE(p_visit_start_time, visit_start_time)
        WHERE id = p_request_id;
    END IF;

    RETURN FOUND;
END;
$$ LANGUAGE plpgsql;

-- Функция добавления в черный список
CREATE OR REPLACE FUNCTION kp.add_to_blacklist(
    p_passport_series VARCHAR,
    p_passport_number VARCHAR,
    p_last_name VARCHAR,
    p_first_name VARCHAR,
    p_middle_name VARCHAR,
    p_reason TEXT,
    p_added_by INT
)
RETURNS INT AS $$
DECLARE
    new_id INT;
BEGIN
    INSERT INTO kp.blacklist (
        passport_series, passport_number, last_name, first_name, middle_name, reason, added_by, added_at
    ) VALUES (
        p_passport_series, p_passport_number, p_last_name, p_first_name, p_middle_name, p_reason, p_added_by, CURRENT_TIMESTAMP
```

```
    )
    RETURNING id INTO new_id;

    RETURN new_id;
EXCEPTION
    WHEN unique_violation THEN
        RETURN -1;
END;
$$ LANGUAGE plpgsql;

-- Функция установки времени убытия
CREATE OR REPLACE FUNCTION kp.set_departure_time(
    p_request_id INT,
    p_request_type VARCHAR
)
RETURNS BOOLEAN AS $$
BEGIN
    IF p_request_type = 'Личная' THEN
        UPDATE kp.personal_requests
        SET visit_end_time = CURRENT_TIMESTAMP
        WHERE id = p_request_id;
    ELSE
        UPDATE kp.group_requests
        SET visit_end_time = CURRENT_TIMESTAMP
        WHERE id = p_request_id;
    END IF;

    RETURN FOUND;
END;
$$ LANGUAGE plpgsql;

-- Функция отправки уведомления
CREATE OR REPLACE FUNCTION kp.send_notification(
    p_user_id INT,
    p_request_id INT,
    p_request_type VARCHAR,
    p_message TEXT
)
RETURNS INT AS $$
DECLARE
    notification_id INT;
BEGIN
    INSERT INTO kp.notifications (user_id, request_id, request_type, message, created_at)
    VALUES (p_user_id, p_request_id, p_request_type, p_message, CURRENT_TIMESTAMP)
    RETURNING id INTO notification_id;

    RETURN notification_id;
```

```
END;
$$ LANGUAGE plpgsql;

-- ==================================================
-- 14. ТРИГГЕРЫ
-- ==================================================

-- Триггер для автоматического формирования логина
CREATE OR REPLACE FUNCTION kp.generate_visitor_login()
RETURNS TRIGGER AS $$
DECLARE
    login_prefix VARCHAR;
    at_position INT;
    new_login VARCHAR;
BEGIN
    at_position := POSITION('@' IN NEW.email);
    IF at_position > 0 THEN
        login_prefix := LEFT(NEW.email, at_position - 1);
    ELSE
        login_prefix := NEW.email;
    END IF;

    new_login := login_prefix || '_' || NEW.id;

    UPDATE kp.users SET login = new_login WHERE id = NEW.id;

    INSERT INTO kp.notifications (user_id, message, created_at)
    VALUES (NEW.id, CONCAT('Ваш логин: ', new_login), NOW());

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tr_generate_visitor_login ON kp.users;
CREATE TRIGGER tr_generate_visitor_login
    AFTER INSERT ON kp.users
    FOR EACH ROW
    EXECUTE FUNCTION kp.generate_visitor_login();

-- Триггер для автоматического добавления в черный список
CREATE OR REPLACE FUNCTION kp.auto_add_to_blacklist()
RETURNS TRIGGER AS $$
DECLARE
    reject_count INT;
    passport_data RECORD;
BEGIN
    IF NEW.status_id = 3 AND NEW.rejection_reason LIKE '%недостоверных данных%' THEN
```

```
    IF TG_TABLE_NAME = 'personal_requests' THEN
      SELECT
        NEW.passport_series AS p_series,
        NEW.passport_number AS p_number,
        NEW.last_name AS l_name,
        NEW.first_name AS f_name,
        NEW.middle_name AS m_name
      INTO passport_data;
    ELSE
      SELECT
        NEW.group_leader_passport_series AS p_series,
        NEW.group_leader_passport_number AS p_number,
        NEW.group_leader_last_name AS l_name,
        NEW.group_leader_first_name AS f_name,
        NEW.group_leader_middle_name AS m_name
      INTO passport_data;
    END IF;

    SELECT COUNT(*) INTO reject_count
    FROM (
      SELECT id FROM kp.personal_requests
      WHERE passport_series = passport_data.p_series
        AND passport_number = passport_data.p_number
        AND status_id = 3
        AND rejection_reason LIKE '%недостоверных данных%'
      UNION ALL
      SELECT id FROM kp.group_requests
      WHERE group_leader_passport_series = passport_data.p_series
        AND group_leader_passport_number = passport_data.p_number
        AND status_id = 3
        AND rejection_reason LIKE '%недостоверных данных%'
    ) AS rejected;

    IF reject_count >= 2 THEN
      INSERT INTO kp.blacklist (passport_series, passport_number, last_name, first_name, middle_name, reason)
      VALUES (passport_data.p_series, passport_data.p_number,
          passport_data.l_name, passport_data.f_name, passport_data.m_name,
          'Автоматическое добавление: двукратное указание недостоверных данных при оформлении заявки');
    END IF;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tr_auto_blacklist_personal ON kp.personal_requests;
```

```sql
CREATE TRIGGER tr_auto_blacklist_personal
    AFTER UPDATE OF status_id ON kp.personal_requests
    FOR EACH ROW
    WHEN (OLD.status_id != 3 AND NEW.status_id = 3)
    EXECUTE FUNCTION kp.auto_add_to_blacklist();

DROP TRIGGER IF EXISTS tr_auto_blacklist_group ON kp.group_requests;
CREATE TRIGGER tr_auto_blacklist_group
    AFTER UPDATE OF status_id ON kp.group_requests
    FOR EACH ROW
    WHEN (OLD.status_id != 3 AND NEW.status_id = 3)
    EXECUTE FUNCTION kp.auto_add_to_blacklist();

-- Триггер для проверки превышения времени перемещения
CREATE OR REPLACE FUNCTION kp.check_time_violation()
RETURNS TRIGGER AS $$
DECLARE
    max_minutes INT;
    actual_minutes INT;
    time_diff INTERVAL;
    visitor_name VARCHAR;
BEGIN
    SELECT from_gate_to_department_minutes INTO max_minutes FROM kp.time_config LIMIT 1;

    IF TG_TABLE_NAME = 'personal_requests' THEN
        visitor_name := NEW.last_name || ' ' || NEW.first_name;
        time_diff := NEW.visit_start_time - OLD.visit_start_time;
    ELSE
        visitor_name := NEW.group_leader_last_name || ' ' || NEW.group_leader_first_name;
        time_diff := NEW.visit_start_time - OLD.visit_start_time;
    END IF;

    actual_minutes := EXTRACT(EPOCH FROM time_diff) / 60;

    IF actual_minutes > max_minutes THEN
        INSERT INTO kp.violations (request_id, request_type, visitor_name, violation_type, description)
        VALUES (NEW.id, TG_TABLE_NAME, visitor_name, 'Превышение времени перемещения',
                'Посетитель превысил время перемещения от проходной до подразделения. Лимит: ' || max_minutes || ' мин., Факт: ' || actual_minutes || ' мин.');

        INSERT INTO kp.notifications (user_id, request_id, request_type, message)
        VALUES (NEW.user_id, NEW.id, TG_TABLE_NAME,
                CONCAT('Нарушение времени перемещения! Посетитель ', visitor_name, ' превысил лимит времени.'));
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```sql
DROP TRIGGER IF EXISTS tr_check_time_violation_personal ON kp.personal_requests;
CREATE TRIGGER tr_check_time_violation_personal
    AFTER UPDATE OF visit_start_time ON kp.personal_requests
    FOR EACH ROW
    WHEN (OLD.visit_start_time IS NULL AND NEW.visit_start_time IS NOT NULL)
    EXECUTE FUNCTION kp.check_time_violation();

DROP TRIGGER IF EXISTS tr_check_time_violation_group ON kp.group_requests;
CREATE TRIGGER tr_check_time_violation_group
    AFTER UPDATE OF visit_start_time ON kp.group_requests
    FOR EACH ROW
    WHEN (OLD.visit_start_time IS NULL AND NEW.visit_start_time IS NOT NULL)
    EXECUTE FUNCTION kp.check_time_violation();

-- ======================================================
-- 15. ПРОВЕРКА СОЗДАННЫХ ОБЪЕКТОВ
-- ======================================================

DO $$
DECLARE
    table_count INT;
    view_count INT;
    func_count INT;
    trigger_count INT;
BEGIN
    SELECT COUNT(*) INTO table_count FROM information_schema.tables WHERE table_schema = 'kp';
    SELECT COUNT(*) INTO view_count FROM information_schema.views WHERE table_schema = 'kp';
    SELECT COUNT(*) INTO func_count FROM pg_proc p JOIN pg_namespace n ON p.pronamespace = n.oid WHERE n.nspname = 'kp';
    SELECT COUNT(*) INTO trigger_count FROM pg_trigger WHERE tgrelid IN ('kp.users'::regclass, 'kp.personal_requests'::regclass, 'kp.group_requests'::regclass);

    RAISE NOTICE '==========================================';
    RAISE NOTICE 'РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ СКРИПТА';
    RAISE NOTICE '==========================================';
    RAISE NOTICE 'Таблиц создано: %', table_count;
    RAISE NOTICE 'Представлений создано: %', view_count;
    RAISE NOTICE 'Функций создано: %', func_count;
    RAISE NOTICE 'Триггеров создано: %', trigger_count;
    RAISE NOTICE '==========================================';
END$$;

-- ======================================================
-- 16. ВЫВОД ТЕСТОВЫХ ДАННЫХ
-- ======================================================

SELECT '=== КОДЫ АВТОРИЗАЦИИ ===' AS info;
SELECT auth_code, role, e.full_name
```

```
FROM kp.employee_auth ea

JOIN kp.employees e ON e.id = ea.employee_id;


SELECT '=== ТЕСТОВЫЕ ПОЛЬЗОВАТЕЛИ ===' AS info;

SELECT id, email, login, created_at FROM kp.users;


SELECT '=== ЧЕРНЫЙ СПИСОК ===' AS info;

SELECT passport_series, passport_number, last_name, first_name, reason FROM kp.blacklist;


SELECT '=== ЗАЯВКИ ===' AS info;

SELECT request_id, request_type, last_name, first_name, status

FROM kp.view_list_requests

LIMIT 5;
```

2. Исходный код приложений

MainWindow.xaml:

```
<Window x:Class="HranitelPRO.MainWindow"
      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      Title="ХранительПРО" Height="700" Width="1000"
      WindowStartupLocation="CenterScreen" Background="#F0F4F8"
      Icon="C:\Shumailov\Практика\4 день\PassOfficeApp\HranitelPRO\images\icon.ico">

   <Grid>
      <Grid.RowDefinitions>
         <RowDefinition Height="50"/>
         <RowDefinition Height="*"/>
         <RowDefinition Height="25"/>
      </Grid.RowDefinitions>

      <Border Grid.Row="0" Background="#1E3A5F">
         <StackPanel Orientation="Horizontal" Margin="10">
            <Image Source="Images/icon.png" Width="32" Height="32" Margin="0,0,10,0"/>
            <TextBlock Text="ХранительПРО" FontSize="20" FontWeight="Bold" Foreground="White" VerticalAlignment="Center"/>
            <StackPanel x:Name="userPanel" Orientation="Horizontal" Margin="20,0,0,0" Visibility="Collapsed">
               <TextBlock x:Name="userEmailText" Foreground="White" VerticalAlignment="Center"/>
               <Button x:Name="logoutButton" Content="Выйти" Click="LogoutButton_Click" Margin="10,0,0,0" Background="#C0392B"
Foreground="White"/>
            </StackPanel>
         </StackPanel>
      </Border>

      <Grid Grid.Row="1" x:Name="mainContent">
         <!-- Форма авторизации посетителя -->
         <StackPanel x:Name="loginPanel" Width="350" HorizontalAlignment="Center" VerticalAlignment="Center">
            <TextBlock Text="Авторизация" FontSize="24" FontWeight="Bold" HorizontalAlignment="Center" Margin="0,0,0,20"/>
            <TextBox x:Name="loginEmail" Margin="0,0,0,10" Height="35"/>
            <PasswordBox x:Name="loginPassword" Margin="0,0,0,10" Height="35"/>
            <Button x:Name="loginButton" Content="Войти" Click="LoginButton_Click" Background="#27AE60" Foreground="White"
Height="40" Margin="0,0,0,10"/>

            <TextBlock HorizontalAlignment="Center" Margin="0,10">
               <Hyperlink x:Name="registerLink" Click="RegisterLink_Click">Зарегистрироваться</Hyperlink>
            </TextBlock>

            <Separator Margin="0,15" Height="1" Background="#D0D5DD"/>
```

```
            <Button x:Name="employeeLoginButton" Content="Вход для сотрудников" Click="EmployeeLoginButton_Click"
                Background="#3498DB" Foreground="White" Height="40" Margin="0,10,0,0"/>

            <TextBlock x:Name="loginErrorText" Foreground="Red" HorizontalAlignment="Center" Margin="0,10,0,0"
TextWrapping="Wrap"/>
        </StackPanel>

        <!-- Форма регистрации -->
        <StackPanel x:Name="registerPanel" Width="350" HorizontalAlignment="Center" VerticalAlignment="Center"
Visibility="Collapsed">
            <TextBlock Text="Регистрация" FontSize="24" FontWeight="Bold" HorizontalAlignment="Center" Margin="0,0,0,20"/>
            <TextBox x:Name="regEmail" Margin="0,0,0,10" Height="35"/>
            <PasswordBox x:Name="regPassword" Margin="0,0,0,5" Height="35"/>
            <TextBlock Text="8+ символов, Aa, 1, @" FontSize="10" Foreground="Gray" Margin="0,0,0,10"/>
            <PasswordBox x:Name="regConfirmPassword" Margin="0,0,0,10" Height="35"/>
            <Button x:Name="registerButton" Content="Зарегистрироваться" Click="RegisterButton_Click" Background="#27AE60"
Foreground="White" Height="40" Margin="0,0,0,10"/>

            <TextBlock HorizontalAlignment="Center" Margin="0,10">
              <Hyperlink x:Name="backToLoginLink" Click="BackToLoginLink_Click">Войти</Hyperlink>
            </TextBlock>

            <TextBlock x:Name="registerErrorText" Foreground="Red" HorizontalAlignment="Center" Margin="0,10,0,0"
TextWrapping="Wrap"/>
        </StackPanel>

        <!-- Основное приложение -->
        <Grid x:Name="appPanel" Visibility="Collapsed">
            <Grid.ColumnDefinitions>
              <ColumnDefinition Width="200"/>
              <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>
            <Border Grid.Column="0" Background="#1E3A5F">
              <StackPanel Margin="10">
                <Button x:Name="personalButton" Content="Личная заявка" Click="PersonalButton_Click" Height="40" Margin="0,5"
Background="#3498DB" Foreground="White"/>
                <Button x:Name="groupButton" Content="Групповая заявка" Click="GroupButton_Click" Height="40" Margin="0,5"
Background="#27AE60" Foreground="White"/>
                <Button x:Name="myRequestsButton" Content="Мои заявки" Click="MyRequestsButton_Click" Height="40" Margin="0,5"
Background="#F39C12" Foreground="White"/>
              </StackPanel>
            </Border>
            <Frame x:Name="contentFrame" Grid.Column="1" NavigationUIVisibility="Hidden"/>
        </Grid>
      </Grid>

      <Border Grid.Row="2" Background="#1E3A5F">
        <TextBlock Text="© 2024 ХранительПРО" Foreground="#7F8C8D" FontSize="10" HorizontalAlignment="Center"
VerticalAlignment="Center"/>
      </Border>
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using Microsoft.Extensions.Configuration;
using System;
using System.IO;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;

namespace HranitelPRO
{
    public partial class MainWindow : Window
    {
        private DatabaseService _db;
        private User _currentUser;

        public MainWindow()
        {
            InitializeComponent();

            // Исправленная строка подключения с параметром Include Error Detail
            string connectionString = "Host=localhost;Port=5432;Database=KhranitelPro;Username=postgres;Password=slon12;Include Error
Detail=true";
            _db = new DatabaseService(connectionString);
        }

        private async void LoginButton_Click(object sender, RoutedEventArgs e)
        {
            string email = loginEmail.Text.Trim();
            string password = loginPassword.Password;

            if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password))
            {
                loginErrorText.Text = "Заполните все поля";
                return;
            }

            var user = await _db.LoginUser(email, password);
            if (user != null)
            {
                _currentUser = user;
                loginPanel.Visibility = Visibility.Collapsed;
                appPanel.Visibility = Visibility.Visible;
                userPanel.Visibility = Visibility.Visible;
                userEmailText.Text = user.email;
                PersonalButton_Click(null, null);
            }
            else
            {
                loginErrorText.Text = "Неверный email или пароль\n\nТест: test@example.com / Test123!";
            }
        }

        private async void RegisterButton_Click(object sender, RoutedEventArgs e)
        {
            string email = regEmail.Text.Trim();
            string password = regPassword.Password;
            string confirm = regConfirmPassword.Password;

            if (!Regex.IsMatch(email, @"^[^@\s]+@[^@\s]+\.[^@\s]+$"))
            {
                registerErrorText.Text = "Неверный email";
                return;
            }

            if (password.Length < 8 || !Regex.IsMatch(password, @"[A-Z]") || !Regex.IsMatch(password, @"[a-z]") || !Regex.IsMatch(password, @"[0-
9]") || !Regex.IsMatch(password, @"[!@#$%^&*()]"))
            {
                registerErrorText.Text = "Пароль: 8+ символов, Aa, 1, @";
                return;
            }

            if (password != confirm)
```

```
        {
            registerErrorText.Text = "Пароли не совпадают";
            return;
        }

        bool success = await _db.RegisterUser(email, password);
        if (success)
        {
            MessageBox.Show("Регистрация успешна!", "Успех");
            ShowLoginPanel();
        }
        else
        {
            registerErrorText.Text = "Email уже существует";
        }
    }

    private void EmployeeLoginButton_Click(object sender, RoutedEventArgs e)
    {
        var authWindow = new AuthWindow(_db);
        authWindow.Show();
        this.Hide();
    }

    private void ShowLoginPanel()
    {
        loginPanel.Visibility = Visibility.Visible;
        registerPanel.Visibility = Visibility.Collapsed;
        appPanel.Visibility = Visibility.Collapsed;
        userPanel.Visibility = Visibility.Collapsed;
    }

    private void RegisterLink_Click(object sender, RoutedEventArgs e)
    {
        loginPanel.Visibility = Visibility.Collapsed;
        registerPanel.Visibility = Visibility.Visible;
    }

    private void BackToLoginLink_Click(object sender, RoutedEventArgs e)
    {
        ShowLoginPanel();
    }

    private void LogoutButton_Click(object sender, RoutedEventArgs e)
    {
        _currentUser = null;
        ShowLoginPanel();
    }

    private void PersonalButton_Click(object sender, RoutedEventArgs e)
    {
        contentFrame.Navigate(new Pages.PersonalRequestPage(_db, _currentUser.id));
    }

    private void GroupButton_Click(object sender, RoutedEventArgs e)
    {
        contentFrame.Navigate(new Pages.GroupRequestPage(_db, _currentUser.id));
    }

    private async void MyRequestsButton_Click(object sender, RoutedEventArgs e)
    {
        var requests = await _db.GetUserRequests(_currentUser.id);
        contentFrame.Navigate(new Pages.MyRequestsPage(requests));
    }
  }
}
```

3. Ссылка на Git-репозиторий

https://github.com/chma-1-sigma/-1.git

№	Сущность	Описание
1	**Пользователи**	Хранит данные о зарегистрированных посетителях (email, логин, пароль). Обеспечивает вход в систему и идентификацию пользователя.
2	**Личные заявки**	Содержит информацию о персональных заявках на посещение: ФИО посетителя, паспортные данные, даты посещения, цель и статус рассмотрения.
3	**Групповые заявки**	Хранит заявки на коллективное посещение: данные руководителя группы, сроки посещения, цель и связь со списком участников группы.
4	**Черный список**	Содержит данные о нежелательных посетителях (паспорт, ФИО, причина). Автоматически отклоняет заявки от лиц, нарушавших правила.
5	**Авторизация сотрудников**	Хранит коды доступа для сотрудников и их роли (общий отдел, охрана, подразделение). Обеспечивает разграничение прав доступа.